

UNIT 3

Advanced Node.js and Database

Topics Covered:

- Introduction to NoSQL Databases
- MongoDB System Overview
- Basic Querying with MongoDB Shell
- MongoDB CRUD Operations using Compass
- Request Body Parsing in Express
- NodeJS – MongoDB Connection (CRUD via Node.js)
- Handling SQL Databases from NodeJS (MySQL + Node.js)
- Handling Cookies in NodeJS
- Handling User Authentication with NodeJS

1. Introduction to NoSQL Databases

1.1 What is a Database?

Before diving into NoSQL, it is essential to understand what a database is. A database is an organised collection of structured information or data, stored electronically on a computer system. Databases are managed by software systems called Database Management Systems (DBMS). Almost every modern application — whether it is a social media platform, an e-commerce website, a banking system, or a hospital management portal — relies on a database to store, retrieve, and manage data efficiently.

Definition: Database

A database is an organised, structured collection of data stored and managed electronically, allowing efficient insertion, retrieval, updating, and deletion of data. A Database Management System (DBMS) is software that provides the interface between the user and the stored data.

1.2 Types of Databases — SQL vs NoSQL

Broadly, databases are classified into two major categories:

- SQL Databases (Relational Databases)
- NoSQL Databases (Non-Relational Databases)

Feature	SQL (Relational)	NoSQL (Non-Relational)
Data Storage	Tables with rows and columns	Documents, key-value pairs, graphs, columns
Schema	Fixed schema – all rows must follow the same structure	Flexible / Dynamic schema – each document can have different fields
Query Language	Structured Query Language (SQL)	Varies by database (MongoDB uses BSON/JSON queries)
Scalability	Vertical scaling (upgrade server hardware)	Horizontal scaling (add more servers)
Transactions	Strong ACID compliance	Eventual consistency in most cases (some support ACID)
Best For	Structured data with complex relationships (banking, ERP)	Large volumes of unstructured / semi-structured data (social media, IoT)
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Redis, Cassandra, CouchDB, Firebase

1.3 What is NoSQL?

Definition: NoSQL Database

NoSQL (originally meaning 'Non SQL' or 'Not Only SQL') refers to a broad category of database management systems that do not use the traditional relational table-based model. NoSQL databases are designed to handle large volumes of unstructured or semi-structured data, offer flexible schemas, scale horizontally across many servers, and deliver high performance for specific use cases. Common NoSQL database types include Document Stores, Key-Value Stores, Column-Family Stores, and Graph Databases.

1.3.1 Types of NoSQL Databases

Type	Description	Example	Use Case
Document Store	Stores data as JSON/BSON documents. Each document can have its own structure.	MongoDB, CouchDB	User profiles, content
Key-Value Store	Simple pairs of key and value. Extremely fast lookups.	Redis, DynamoDB	Caching, sessions
Column-Family	Stores data in columns rather than rows. Good for analytics.	Cassandra, HBase	Time-series, logs
Graph Database	Stores data as nodes and edges representing relationships.	Neo4j, Amazon Neptune	Social networks, recommendations

1.4 Advantages of NoSQL Databases

- **Flexible Schema:** NoSQL databases allow each record to have a different structure. There is no need to define a fixed schema upfront. New fields can be added at any time without modifying existing records.
- **Horizontal Scalability:** NoSQL databases are designed to scale out across many servers (horizontal scaling) rather than scaling up by upgrading a single server. This makes them ideal for handling millions of users.
- **High Performance:** Because NoSQL databases avoid costly JOIN operations (which are common in relational databases), they can retrieve data much faster for specific access patterns.
- **Handles Unstructured Data:** NoSQL can store unstructured data such as social media posts, multimedia metadata, sensor data, and log files that do not fit neatly into rows and columns.
- **High Availability:** Many NoSQL databases support automatic data replication across multiple servers, ensuring that the system continues to work even if one server fails.

2. MongoDB System Overview

2.1 What is MongoDB?

Definition: MongoDB

MongoDB is a free, open-source, cross-platform, document-oriented NoSQL database management system. It stores data in flexible, JSON-like documents called BSON (Binary JSON) documents, where fields can vary from document to document and the data structure can be changed over time. MongoDB is designed for scalability, performance, and ease of development. It was developed by MongoDB Inc. (originally 10gen) and first released in 2009. The name 'MongoDB' comes from the word 'humongous', reflecting its ability to handle very large amounts of data.

2.2 Key Terminology — MongoDB vs SQL Comparison

Understanding the MongoDB terminology in relation to SQL concepts you already know is essential:

SQL Concept	MongoDB Equivalent	Description
Database	Database	Top-level container for data
Table	Collection	A group of related documents
Row / Record	Document	A single data record stored as BSON
Column / Field	Field	A key-value pair inside a document
Primary Key	_id field	Unique identifier auto-generated for each document
JOIN	\$lookup (Aggregation)	Combining data from multiple collections
INDEX	Index	Improves query performance

2.3 What is a Document?

Definition: Document (in MongoDB)

A document in MongoDB is the basic unit of data, equivalent to a row in a relational database. A document is stored in BSON (Binary JSON) format, which is a binary-encoded serialisation of JSON-like objects. A document consists of field-value pairs, where field names are strings and values can be of any BSON data type including strings, numbers, booleans, arrays, nested documents, dates, and null. Each document in a collection automatically receives a unique identifier stored in the '_id' field.

Example of a MongoDB Document:

```
{
  _id: ObjectId('64f1a2b3c4d5e6f7a8b9c0d1'),
  name: 'Priya Sharma',
  age: 22,
  city: 'Surat',
  enrolled: true,
  subjects: ['FSD', 'DBMS', 'OS'],
  address: {
    street: 'Ring Road',
    pincode: 395007
  }
}
```

2.4 What is a Collection?

Definition: Collection (in MongoDB)

A collection is a group of MongoDB documents, equivalent to a table in a relational database. Unlike SQL tables, a MongoDB collection does not enforce a schema — documents within the same collection can have different fields and data types. Collections are created automatically when you first insert a document into them. Collections are stored within a database.

2.5 What is BSON?

Definition: BSON (Binary JSON)

BSON stands for Binary JSON. It is the binary-encoded format used internally by MongoDB to store and transfer documents. While JSON is text-based and human-readable, BSON is a binary representation that is more efficient to encode, decode, and traverse. BSON supports additional data types that JSON does not natively support, such as Date, Binary, ObjectId, and Int32/Int64. When you interact with MongoDB (through Compass or Node.js), you work with JSON-like syntax, and MongoDB handles the BSON conversion internally.

2.6 The `_id` Field

Every document in MongoDB must have an `_id` field that acts as its unique primary key. If you do not provide an `_id` when inserting a document, MongoDB automatically generates one using the `ObjectId` data type.

Definition: ObjectId

An ObjectId is a 12-byte BSON type that MongoDB uses as the default value for the `_id` field. The 12 bytes are composed of: a 4-byte timestamp (seconds since Unix epoch), a 5-byte random value (unique to the machine and process), and a 3-byte incrementing counter. This design ensures that ObjectIds are globally unique and naturally ordered by creation time. Example: `ObjectId('64f1a2b3c4d5e6f7a8b9c0d1')`

2.7 MongoDB Tools

Tool	Description
MongoDB Compass	Official GUI (Graphical User Interface) tool for MongoDB. Allows you to visually create databases, collections, insert/update/delete documents, and run queries without writing shell commands.
MongoDB Shell (mongosh)	Command-line interface for interacting with MongoDB. Used to run queries and administrative commands directly.
MongoDB Atlas	Cloud-hosted MongoDB service. Allows you to run MongoDB on the cloud without managing servers.
Node.js Driver	Official npm package (mongodb) for connecting and interacting with MongoDB from a Node.js application.

3. MongoDB CRUD Operations

CRUD stands for Create, Read, Update, and Delete — the four fundamental operations that can be performed on any database. In MongoDB, these operations are performed on documents within collections.

CRUD	Operation	MongoDB Method	Description
C	Create / Insert	insertOne(), insertMany()	Add new documents to a collection
R	Read / Find	find(), findOne()	Retrieve documents from a collection
U	Update	updateOne(), updateMany()	Modify existing documents
D	Delete	deleteOne(), deleteMany()	Remove documents from a collection

3.1 CREATE Operations — Inserting Documents

3.1.1 insertOne()

Definition: insertOne()

insertOne() is a MongoDB method used to insert a single document into a collection. If the collection does not exist, MongoDB automatically creates it. The method takes one argument — the document to insert — as a JavaScript object. It returns a result object containing the insertedId of the newly created document.

Syntax:

```
db.collectionName.insertOne({ field1: value1, field2: value2, ... })
```

Example — Inserting a book document:

```
db.books.insertOne({
  title: 'The Alchemist',
  author: 'Paulo Coelho',
  genre: 'Fiction',
  price: 250,
  available: true,
  pages: 208,
  year: 1988,
  language: 'English'
})
```

Expected Output:

```
{
  acknowledged: true,
  insertedId: ObjectId('64f1a2b3c4d5e6f7a8b9c0d1')
}
```

3.1.2 insertMany()

Definition: insertMany()

`insertMany()` is used to insert multiple documents into a collection in a single operation. It takes an array of document objects as its argument. This is more efficient than calling `insertOne()` multiple times in a loop because it sends all documents to the server in a single network request.

Syntax:

```
db.collectionName.insertMany([ {doc1}, {doc2}, {doc3}, ... ])
```

Example — Inserting multiple books:

```
db.books.insertMany([
  { title: 'Wings of Fire', author: 'A.P.J. Abdul Kalam', genre:
    'Biography',
    price: 180, available: true, pages: 196, year: 1999, language: 'Hindi'
  },
  { title: 'Harry Potter', author: 'J.K. Rowling', genre: 'Fantasy',
    price: 450, available: true, pages: 332, year: 1997, language:
    'English' },
  { title: 'Rich Dad Poor Dad', author: 'Robert Kiyosaki', genre: 'Self-
    Help',
    price: 320, available: false, pages: 272, year: 1997, language:
    'English' },
  { title: 'Godan', author: 'Munshi Premchand', genre: 'Fiction',
    price: 150, available: false, pages: 350, year: 1936, language:
    'Hindi' },
])
```


3.2 READ Operations — Finding Documents

3.2.1 find()

Definition: find()

`find()` retrieves all documents from a collection that match a given query filter. When called with no arguments (empty object `{}`), it returns all documents. The result is returned as a cursor — a pointer to the result set — which can be converted to an array using `.toArray()` in Node.js or automatically displayed in MongoDB Compass.

Syntax:

```
db.collectionName.find( { query } )
```

Examples:

```
// Fetch ALL documents
db.books.find({})

// Fetch all books where available is true
db.books.find({ available: true })

// Fetch all books where price > 300 (using $gt operator)
db.books.find({ price: { $gt: 300 } })

// Fetch books in Fiction OR Fantasy genre (using $in operator)
db.books.find({ genre: { $in: ['Fiction', 'Fantasy'] } })

// Fetch Hindi books with price < 200 (two conditions)
db.books.find({ language: 'Hindi', price: { $lt: 200 } })
```

3.2.2 findOne()

Definition: findOne()

`findOne()` retrieves a single document that matches the given query filter. If multiple documents match the filter, `findOne()` returns only the first one encountered in natural order. If no document matches, it returns null. It is used when you know there is only one matching document, or when you only need one result.

```
// Find the book titled 'The Alchemist'
db.books.findOne({ title: 'The Alchemist' })

// Find one available book
db.books.findOne({ available: true })
```

3.2.3 MongoDB Query Operators

MongoDB provides powerful query operators that allow you to perform complex comparisons and logical operations:

Operator	Meaning	Example
\$eq	Equal to	{ price: { \$eq: 250 } } — books with price exactly 250
\$ne	Not equal to	{ genre: { \$ne: 'Fiction' } } — not Fiction
\$gt	Greater than	{ price: { \$gt: 300 } } — price above 300
\$gte	Greater than or equal	{ price: { \$gte: 300 } }
\$lt	Less than	{ price: { \$lt: 200 } } — price below 200
\$lte	Less than or equal	{ price: { \$lte: 200 } }
\$in	Value in array	{ genre: { \$in: ['Fiction','Fantasy'] } }
\$nin	Value NOT in array	{ genre: { \$nin: ['Self-Help'] } }
\$and	All conditions true	{ \$and: [{available:true},{price:{\$gt:200}}] }
\$or	At least one condition	{ \$or: [{genre:'Fiction'},{language:'Hindi'}] }

3.2.4 Sorting, Limiting and Projection

MongoDB allows you to sort, limit, and project (select specific fields from) your query results:

```
// Sort by price ascending (1 = ASC, -1 = DESC)
db.books.find({}).sort({ price: 1 })

// Sort descending and get top 3
db.books.find({}).sort({ price: -1 }).limit(3)

// Projection: show only title, author, price — hide _id
// 1 = include field, 0 = exclude field
db.books.find({}, { title: 1, author: 1, price: 1, _id: 0 })
```

3.3 UPDATE Operations — Modifying Documents

MongoDB provides two primary update methods. Both methods require two arguments: (1) a filter to identify which documents to update, and (2) an update object using update operators like \$set or \$inc.

3.3.1 Update Operators

Operator	Purpose	Example
\$set	Set a field to a specific value. Can also add new fields.	{ \$set: { price: 300, available: true } }
\$inc	Increment (or decrement) a numeric field by a value.	{ \$inc: { price: 50 } } (adds 50 to current price)
\$unset	Remove a field from a document.	{ \$unset: { rating: " } }
\$push	Add an element to an array field.	{ \$push: { tags: 'bestseller' } }

3.3.2 updateOne()

Definition: updateOne()

updateOne() modifies the first document that matches the given filter. If multiple documents match the filter, only the first one is updated. It takes two required arguments: (1) the filter document to identify the target, and (2) the update document using update operators.

```
// Change the price of 'The Alchemist' to 280
db.books.updateOne(
  { title: 'The Alchemist' },
  { $set: { price: 280 } }
)

// Make an unavailable book available
db.books.updateOne(
  { available: false },
  { $set: { available: true } }
)
```

3.3.3 updateMany()

Definition: updateMany()

updateMany() modifies ALL documents that match the given filter. It is used when you need to apply the same change to multiple documents at once. The return value includes matchedCount (how many documents matched) and modifiedCount (how many were actually changed).

```
// Add a 'rating' field with value 0 to ALL documents
db.books.updateMany(
  {},
  { $set: { rating: 0 } }
)

// Increase price by 50 for all Self-Help books
db.books.updateMany(
  { genre: 'Self-Help' },
  { $inc: { price: 50 } }
)
```

3.4 DELETE Operations — Removing Documents

3.4.1 deleteOne()

Definition: deleteOne()

`deleteOne()` removes the first document that matches the given filter from the collection. It is the safest delete method when you want to remove only one specific document. Always use a unique identifier (like `_id` or `title`) in the filter to ensure you delete the correct document.

```
// Insert a temporary dummy document first
db.books.insertOne({ title: 'DUMMY BOOK', author: 'Test', genre: 'Test',
  price: 0, available: false, pages: 0, year: 2024, language: 'English' })

// Now delete only that dummy document
db.books.deleteOne({ title: 'DUMMY BOOK' })
```

3.4.2 deleteMany()

Definition: deleteMany()

`deleteMany()` removes ALL documents that match the given filter from the collection. It is used when you want to clean up multiple records at once. Passing an empty filter (`{}`) to `deleteMany()` will delete ALL documents in the collection — use this with extreme caution.

```
// Delete all books where available is false
db.books.deleteMany({ available: false })

// Verify the deletion by fetching all remaining documents
db.books.find({})
```

4. Request Body Parsing in Express

4.1 What is Request Body Parsing?

When a client (browser or mobile app) sends data to the server via a POST, PUT, or PATCH request, the data is included in the body of the HTTP request. Unlike query parameters (which appear in the URL) or route parameters (which are part of the URL path), request body data is not automatically available in Express.

Definition: Request Body Parsing

Request body parsing is the process by which an Express server reads and interprets the raw incoming data from the body of an HTTP request and makes it accessible through the `req.body` object in route handlers. Without body parsing middleware, `req.body` would be undefined, and the server would not be able to read POST data, form submissions, or JSON payloads sent by the client.

4.2 Method 1 — Using the body-parser Package (Third-Party)

Definition: body-parser

`body-parser` is a separate npm package that was the original and most widely used body-parsing middleware for Node.js and Express. Before Express version 4.16.0, Express did not include built-in body parsing — developers had to install `body-parser` as an external package. Even today, many tutorials, older codebases, and some teachers use `body-parser` explicitly because it is well-known and still fully supported. It provides the same functionality as the built-in Express parsers but is installed and imported separately.

4.2.1 Installing body-parser

```
npm install body-parser
```

4.2.2 Using body-parser for JSON Data

`bodyParser.json()` parses incoming requests with JSON payloads. The parsed data is made available on `req.body`.

```
const express    = require('express');
const bodyParser = require('body-parser');
const app        = express();

// Register body-parser middleware for JSON
// CRITICAL: Must be placed BEFORE any routes that use req.body
app.use(bodyParser.json());
```

```
app.post('/api/student', function(req, res) {
  const data = req.body; // Access parsed JSON body
  console.log(data.name);
  res.json({ message: 'Student received', student: data });
});

app.listen(3000, () => console.log('Server running!'));
```

4.2.3 Using body-parser for HTML Form Data

`bodyParser.urlencoded()` parses incoming requests with URL-encoded payloads — the format used by HTML form submissions (Content-Type: application/x-www-form-urlencoded).

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();

// Parse URL-encoded form data
// extended: true - uses the 'qs' library (supports nested objects)
// extended: false - uses the built-in 'querystring' library (simple)
app.use(bodyParser.urlencoded({ extended: true }));

app.post('/submit-form', function(req, res) {
  const name = req.body.name;
  const email = req.body.email;
  res.send('Received: ' + name + ' - ' + email);
});

app.listen(3000);
```

4.2.4 Using Both Together (JSON + URL-encoded)

In most real-world applications, you register both parsers so the server can handle both types of incoming data:

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();

app.use(bodyParser.json()); // For JSON data
app.use(bodyParser.urlencoded({ extended: true })); // For form data

app.post('/register', function(req, res) {
  console.log(req.body); // Works for both JSON and form submissions
  res.json({ received: req.body });
});

app.listen(3000);
```

4.3 Method 2 — Using Built-in Express Parsers (Modern Way)

From Express version 4.16.0 onwards, Express introduced its own built-in body-parsing middleware — `express.json()` and `express.urlencoded()`. These are built directly into Express and work exactly the same as `body-parser`, so there is no need to install a separate package. This is the recommended approach for all new projects.

4.3.1 `express.json()` — For JSON Data

Used to parse incoming requests whose Content-Type header is `application/json`. This is the most commonly used parser in REST API development when clients send JSON data.

```
const express = require('express');
const app = express();

// Built-in — no separate package needed
// CRITICAL: Must be placed BEFORE any routes that use req.body
app.use(express.json());

app.post('/api/student', function(req, res) {
  const data = req.body; // Access parsed JSON body
  console.log(data.name); // Works because json() middleware ran first
  res.json({ message: 'Student received', student: data });
});

app.listen(3000);
```

4.3.2 express.urlencoded() — For HTML Form Data

Used to parse incoming requests whose Content-Type header is application/x-www-form-urlencoded. This is the encoding used by HTML forms when submitted via the POST method.

```
// Built-in — no separate package needed
// extended: true allows parsing of rich objects and arrays
// extended: false uses the simpler querystring library
app.use(express.urlencoded({ extended: true }));

app.post('/submit-form', function(req, res) {
  const name = req.body.name;
  const email = req.body.email;
  res.send('Received: ' + name + ' - ' + email);
});
```

4.4 body-parser vs Built-in Express — Comparison

Feature	body-parser (npm package)	Express Built-in (express.json/urlencoded)
Installation	npm install body-parser required	No installation — built into Express 4.16+
Import	const bodyParser = require('body-parser')	Already available via express object
JSON parsing	app.use(bodyParser.json())	app.use(express.json())
Form parsing	app.use(bodyParser.urlencoded({extended:true}))	app.use(express.urlencoded({extended:true}))
Result on req.body	Same — parsed JavaScript object	Same — parsed JavaScript object
Express version	Works with all versions of Express	Express 4.16.0 and above only
Usage today	Still valid; seen in older projects and tutorials	Recommended for all new projects

Important

Both body-parser and the built-in Express parsers must be registered using app.use() BEFORE any route handlers that need to access req.body. If placed after routes, req.body will be undefined for those routes. The end result is identical — req.body will contain the parsed data in both cases.

4.5 How Body Parsing Works — Step by Step

1. The client sends an HTTP POST request with data in the request body.
2. Express receives the raw HTTP request with a binary/text body.
3. The body-parsing middleware (body-parser or the built-in `express.json` / `express.urlencoded`) intercepts the request.
4. It reads the raw body data, parses it based on the Content-Type header, and attaches the parsed JavaScript object to `req.body`.
5. It calls `next()` to pass control to the next middleware or route handler.
6. The route handler can now access the parsed data through `req.body`.

5. NodeJS – MongoDB Connection and CRUD

5.1 Setup and Installation

To connect a Node.js application to MongoDB, you need to install the official MongoDB Node.js driver package using npm.

Step 1: Create project and install the driver:

```
mkdir my-mongodb-app
cd my-mongodb-app
npm init -y
npm install mongodb
```

Step 2: Create a file named index.js

5.2 Connecting to MongoDB from Node.js

The MongoDB driver provides the MongoClient class to establish a connection to a MongoDB server.

```
const { MongoClient } = require('mongodb');

// Connection URL – copy from MongoDB Compass
const url = 'mongodb://localhost:27017';
const client = new MongoClient(url);

async function connectDB() {
  try {
    await client.connect();
    console.log('Connected to MongoDB...!');

    // Select (or create) a database
    const db = client.db('Student_MCA');

    // Create a collection
    await db.createCollection('users');
    console.log('Collection created...!');

    const collection = db.collection('users');

    // --- Perform CRUD operations here ---

  } catch (err) {
    console.error('Error:', err);
  } finally {
```

```
    await client.close(); // Always close the connection
  }
}

connectDB();
```

5.3 INSERT Operations via Node.js

5.3.1 Insert One Document

```
const result = await collection.insertOne({
  name: 'Meet Patel',
  age: 22,
  city: 'Surat',
  status: 'active'
});
console.log('Inserted ID:', result.insertedId);
// Output: Inserted ID: 64f1a2b3c4d5e6f7a8b9c0d1
```

5.3.2 Insert Many Documents

```
const users = [
  { name: 'Raj Shah', age: 21, city: 'Ahmedabad', status: 'active' },
  { name: 'Priya Mehta', age: 23, city: 'Vadodara', status: 'inactive' },
  { name: 'Amit Joshi', age: 25, city: 'Surat', status: 'active' }
];

const manyResult = await collection.insertMany(users);
console.log('Inserted Count:', manyResult.insertedCount);
// Output: Inserted Count: 3
```

5.4 FIND Operations via Node.js

5.4.1 Find One Document

```
const user = await collection.findOne({ name: 'Raj Shah' });
if (user) {
  console.log('Found:', user);
} else {
  console.log('User not found.');
}
// Output: Found: { _id: ..., name: 'Raj Shah', age: 21, ... }
```

5.4.2 Find All Documents

```
// .toArray() converts the cursor to a JavaScript array
const allUsers = await collection.find().toArray();
console.log('All users:', allUsers);
```

5.4.3 Filter Documents

```
// Find only active users
const activeUsers = await collection.find({ status: 'active' }).toArray();
console.log('Active users:', activeUsers);

// Find users aged above 20
const aboveAge = await collection.find({ age: { $gt: 20 } }).toArray();
console.log('Users above age 20:', aboveAge);
```

5.5 UPDATE Operations via Node.js

5.5.1 Update One Document

```
const r1 = await collection.updateOne(
  { name: 'Priya Mehta' }, // filter
  { $set: { city: 'Ahmedabad' } } // update
);
console.log('Matched:', r1.matchedCount, 'Modified:', r1.modifiedCount);
// Output: Matched: 1 Modified: 1
```

5.5.2 Update Many Documents

```
const r2 = await collection.updateMany(
  { city: 'Surat' },
  { $set: { status: 'active' } }
);
console.log(`${r2.modifiedCount} records updated`);
```

5.6 DELETE Operations via Node.js

5.6.1 Delete One Document

```
const r1 = await collection.deleteOne({ name: 'Meet Patel' });
console.log('Deleted:', r1.deletedCount);
// Output: Deleted: 1
```

5.6.2 Delete Many Documents

```
const r2 = await collection.deleteMany({ status: 'inactive' });
console.log(`${r2.deletedCount} docs deleted`);
```

5.7 Quick Reference — MongoDB Node.js Driver

Operation	Method	Returns
Insert one	collection.insertOne({})	{ insertedId }
Insert many	collection.insertMany([])	{ insertedCount, insertedIds }
Find one	collection.findOne({})	Document or null
Find all	collection.find().toArray()	Array of documents
Update one	collection.updateOne({}, {})	{ matchedCount, modifiedCount }
Update many	collection.updateMany({}, {})	{ matchedCount, modifiedCount }
Delete one	collection.deleteOne({})	{ deletedCount }
Delete many	collection.deleteMany({})	{ deletedCount }

6. Handling SQL Databases from NodeJS — MySQL

6.1 Introduction

In addition to MongoDB (a NoSQL database), Node.js can also connect to and interact with relational SQL databases such as MySQL. This is important for full-stack developers who work on projects that use traditional relational databases. MySQL is one of the most widely used open-source relational database management systems in the world, used in applications like WordPress, Facebook (early days), and many enterprise systems.

6.2 Pre-Requirements and Setup

6.2.1 Required Tools

- Node.js installed on your computer
- XAMPP installed — provides MySQL server and phpMyAdmin (a web-based MySQL management interface). Download from apachefriends.org. **Alternatively, you can install MySQL separately and use it as a database if you don't want to use XAMPP.**
- A code editor such as VS Code

6.2.2 Installing the MySQL Driver

```
// Create project folder
mkdir MYSQL_Node_CRUD
cd MYSQL_Node_CRUD
npm init -y

// Install the mysql2 package (faster and more feature-rich than mysql)
npm install mysql2
```

mysql vs mysql2

Always use the mysql2 package rather than the older mysql package. mysql2 supports Promises and async/await natively, has better performance, and includes built-in SQL injection protection through prepared statements.

6.2.3 Creating the Database and Table in phpMyAdmin

7. Open XAMPP Control Panel and click Start next to Apache and MySQL.
8. Open your browser and go to <http://localhost/phpmyadmin>
9. Click New in the left sidebar and create a database named users.
10. Select the users database, go to the SQL tab, and run the following query:

```
CREATE TABLE tbl_users (  
  id      INT AUTO_INCREMENT PRIMARY KEY,  
  name    VARCHAR(100),  
  email   VARCHAR(100)  
);
```

6.3 Creating the Database Connection — db.js

Create a file named db.js in your project folder. This file will contain the database connection and all CRUD operations.

```
const mysql = require('mysql2');  
  
const connection = mysql.createConnection({  
  host:      'localhost',  
  user:      'root',  
  password:  '', // Leave empty if no password set in XAMPP  
  database:  'users'  
});  
  
connection.connect((err) => {  
  if (err) {  
    console.error('Error:', err);  
    return;  
  }  
  console.log('Connected to MySQL...!');  
});
```

6.4 INSERT Operations

6.4.1 Insert a Single User

```
const sql = 'INSERT INTO tbl_users (name, email) VALUES (?, ?)';  
const values = ['Meet', 'meet@gmail.com'];  
  
connection.query(sql, values, (err, result) => {  
  if (err) { console.error('Error:', err); return; }  
  console.log('Inserted ID:', result.insertId);  
});  
  
// Run the file: node db.js  
// Output: Inserted ID: 1
```

6.4.2 Insert Multiple Users

```
const usersData = [
  ['Jeet', 'jeet@gmail.com'],
  ['Rahul', 'rahul@gmail.com'],
  ['Akash', 'akash@gmail.com'],
];

const sql = 'INSERT INTO tbl_users (name, email) VALUES ?';

connection.query(sql, [usersData], (err, result) => {
  if (err) { console.error('Error:', err); return; }
  console.log('Users Inserted:', result.affectedRows);
});
```

6.5 SELECT Operation — Fetch All Users

```
connection.query('SELECT * FROM tbl_users', (err, results) => {
  if (err) { console.error('Error:', err); return; }
  console.log('All Users:', results);
});

// results is an array of JavaScript objects
// Example: [ { id: 1, name: 'Meet', email: 'meet@gmail.com' }, ... ]
```

6.6 UPDATE Operation

```
const sql = 'UPDATE tbl_users SET name = ? WHERE email = ?';
const values = ['Jeetss', 'jeet@gmail.com'];

connection.query(sql, values, (err, result) => {
  if (err) { console.error('Error:', err); return; }
  console.log('Updated rows:', result.affectedRows);
});
```

6.7 DELETE Operation

```
const sql = 'DELETE FROM tbl_users WHERE name = ?';
const values = ['Jeetss'];

connection.query(sql, values, (err) => {
  if (err) { console.error('Error:', err); return; }
  console.log('User Deleted..!');
});
```


6.8 Closing the Connection

Always close the database connection at the end of your script to prevent resource leaks:

```
connection.end();
```

6.9 connection.query() vs connection.execute() — SQL Injection

Definition: SQL Injection

SQL Injection is a code injection attack in which malicious SQL code is inserted into a query via user input. If not prevented, an attacker could manipulate database queries to access, modify, or delete data they are not authorised to access.

	connection.query() with ?	connection.execute() with ?
How it works	mysql2 replaces ? with values on Node.js side before sending one complete SQL string to MySQL.	SQL and values are sent to MySQL separately. MySQL compiles SQL first, then fills values safely.
SQL Injection	No protection if you build SQL manually (without ?). Using ? provides basic safety.	Built-in SQL injection protection. MySQL compiles SQL once and safely fills values.
Recommendation	Acceptable for learning. Always use ? placeholders.	Preferred for production applications.

6.10 Quick Reference — MySQL Node.js CRUD

Step	Operation	SQL Keyword / Code
5	Add one user	INSERT INTO tbl_users (name, email) VALUES (?, ?)
6	Add many users	INSERT INTO tbl_users (name, email) VALUES ? (with array)
7	Change user data	UPDATE tbl_users SET name = ? WHERE email = ?
8	Remove a user	DELETE FROM tbl_users WHERE name = ?
9	Show all users	SELECT * FROM tbl_users

7. Handling Cookies in NodeJS

7.1 What is a Cookie?

Definition: Cookie

A cookie is a small piece of data — a key-value pair — that a web server sends to the client's browser in an HTTP response. The browser stores this data on the user's computer and automatically includes it in every subsequent HTTP request to the same server. Cookies are used to maintain state across multiple HTTP requests, since HTTP itself is a stateless protocol (each request is independent and the server does not remember previous interactions).

7.2 Why Do We Need Cookies?

HTTP is a stateless protocol. This means each request from a browser to a server is completely independent — the server does not remember who you are or what you did previously. Cookies solve this problem by providing a mechanism for the server to store a small amount of data on the client's browser, which is then sent back to the server with every request. This allows the server to 'remember' the user across multiple requests.

Common use cases for cookies:

- Keeping users logged in (remember me functionality)
- Storing user preferences (theme, language, font size)
- Tracking shopping cart contents in e-commerce
- Analytics and tracking user behaviour
- Session management (storing a session ID)

7.3 Installing cookie-parser Middleware

To work with cookies easily in Express, we use the cookie-parser middleware package:

```
npm install cookie-parser
```

7.4 Setting Up and Using Cookies in Express

```
const express = require('express');
const cookieParser = require('cookie-parser');
const app = express();

// Register the cookie-parser middleware
// This parses incoming cookies and makes them available via req.cookies
app.use(cookieParser());

// ROUTE 1: Set a cookie
app.get('/set-cookie', function(req, res) {
```

```
// res.cookie(name, value, options)
res.cookie('username', 'PriyaSharma', {
  maxAge: 24 * 60 * 60 * 1000, // Expires in 1 day (in milliseconds)
  httpOnly: true,               // Cookie not accessible via JavaScript
  secure: false,                // Set to true for HTTPS only
});

res.send('Cookie has been set!');
});

// ROUTE 2: Read a cookie
app.get('/get-cookie', function(req, res) {
  const username = req.cookies.username; // Read cookie by name
  if (username) {
    res.send('Welcome back, ' + username + '!');
  } else {
    res.send('No cookie found. Please visit /set-cookie first.');
```

```
  }
});

// ROUTE 3: Delete a cookie
app.get('/clear-cookie', function(req, res) {
  res.clearCookie('username'); // Remove the cookie by name
  res.send('Cookie has been deleted!');
});

app.listen(3000, () => console.log('Cookie demo running!'));
```

7.5 Cookie Options Explained

Option	Type / Default	Description
maxAge	Number (milliseconds)	How long the cookie lasts. After this time, browser deletes it automatically.
expires	Date object	Specific expiry date/time for the cookie.
httpOnly	Boolean (false)	If true, cookie is not accessible via client-side JavaScript (document.cookie). Protects against XSS attacks.
secure	Boolean (false)	If true, cookie is only sent over HTTPS connections. Set to true in production.
sameSite	'Strict'/'Lax'/'None'	Controls cross-site request behaviour. 'Strict' = only same site. Prevents CSRF attacks.
path	String (default: '/')	The URL path the cookie applies to. Default '/' means all pages.

7.6 Session Cookies vs Persistent Cookies

	Session Cookie	Persistent Cookie
Duration	Deleted when browser is closed	Survives browser restarts until maxAge/expires
How to create	Do not set maxAge or expires	Set maxAge or expires option
Use case	Temporary state (shopping cart during visit)	Remember me, user preferences

8. Handling User Authentication with NodeJS

8.1 What is Authentication?

Definition: Authentication

Authentication is the process of verifying the identity of a user — confirming that the user is who they claim to be. In web applications, this typically involves the user providing credentials (such as a username and password), which the server verifies against stored data. If the credentials match, the user is considered authenticated and is granted access to protected resources.

 Authentication

Authentication is the process of verifying the identity of a user or system. It ensures that the user is legitimate by validating credentials like passwords, OTPs, or biometrics.

Working

The authentication process verifies the identity of a user before granting access to the system.

- User enters credentials (password, OTP, biometrics)
- System verifies the credentials
- If valid, the user is successfully authenticated

 Authorization

Authorization determines the access rights and permissions of an authenticated user. It decides what resources the user can access and what actions they are allowed to perform.

Working

The system uses authentication and authorization processes to control access and ensure security.

- System checks the user's roles or permissions
- Grants or denies access to resources
- Ensures the user can perform only allowed actions

8.2 Authentication vs Authorisation

	Authentication	Authorisation
Question it answers	Who are you?	What are you allowed to do?
Example	Logging in with username and password	An admin can delete posts; a regular user cannot
Happens	First (before authorisation)	After authentication
Result	Establishes identity	Grants or denies access to resources

8.3 Why Should Passwords Never Be Stored as Plain Text?

Storing passwords as plain text in a database is extremely dangerous. If an attacker gains access to your database, they will have every user's password immediately. The industry-standard solution is password hashing — converting the password into an irreversible, fixed-length string using a cryptographic hash function.

Definition: Password Hashing

Password hashing is a one-way cryptographic process that converts a plain-text password into a fixed-length string (the hash) that cannot be reversed to reveal the original password. When a user logs in, the server hashes the entered password and compares it to the stored hash — if they match, the password is correct. bcrypt is the most widely recommended hashing algorithm for passwords because it is intentionally slow (making brute-force attacks impractical) and includes a built-in 'salt' to prevent rainbow table attacks.

8.4 Installing Required Packages

```
npm install express bcrypt express-session cookie-parser

// bcrypt          - for hashing passwords
// express-session - for managing user sessions
// cookie-parser    - for reading cookies
```

8.5 User Registration with Password Hashing

```
const express = require('express');
const bcrypt  = require('bcrypt');
const app     = express();
app.use(express.json());

// In-memory user store (use a database in production)
const users = [];

// REGISTRATION ROUTE
app.post('/register', async function(req, res) {
  try {
    const { username, password } = req.body;

    // Validate that both fields are provided
    if (!username || !password) {
      return res.status(400).json({ error: 'Username and password required.' });
    }
  }
}
```

```
// Check if user already exists
const existing = users.find(u => u.username === username);
if (existing) {
  return res.status(400).json({ error: 'Username already taken.' });
}

// Hash the password – saltRounds=10 means bcrypt runs 2^10 = 1024
times
// More rounds = slower but more secure
const saltRounds = 10;
const hashedPassword = await bcrypt.hash(password, saltRounds);

// Store user with hashed password (NEVER store plain password)
users.push({ username, password: hashedPassword });

res.status(201).json({ message: 'User registered successfully!' });
} catch (error) {
  res.status(500).json({ error: 'Server error during registration.' });
}
});
```

8.6 User Login — Verifying Password

```
// LOGIN ROUTE
app.post('/login', async function(req, res) {
  try {
    const { username, password } = req.body;

    // Find the user in the database
    const user = users.find(u => u.username === username);
    if (!user) {
      return res.status(401).json({ error: 'Invalid username or password.'
});
    }

    // Compare entered password with stored hash
    // bcrypt.compare() returns true if they match, false if not
    const isValidPassword = await bcrypt.compare(password, user.password);

    if (!isValidPassword) {
      return res.status(401).json({ error: 'Invalid username or password.'
});
    }
  }
});
```

```
// Password is correct – user is authenticated!
res.json({ message: 'Login successful! Welcome, ' + username });

} catch (error) {
  res.status(500).json({ error: 'Server error during login.' });
}
});

app.listen(3000, () => console.log('Auth server running!'));
```

8.7 Session-Based Authentication with express-session

Definition: Session

A session is a server-side mechanism for maintaining state across multiple HTTP requests from the same user. When a user logs in, the server creates a session object and stores it in memory (or a database). A unique session ID is sent to the client's browser as a cookie. On every subsequent request, the browser sends this session ID cookie, allowing the server to look up the session data and identify the user.

```
const express      = require('express');
const bcrypt       = require('bcrypt');
const session      = require('express-session');
const app          = express();

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Configure session middleware
app.use(session({
  secret:      'my_super_secret_key', // Used to sign the session ID
  cookie:      {
    resave:      false, // Do not save session if unmodified
    saveUninitialized: false, // Do not create session until data is stored
  },
  cookie: {
    maxAge:      60 * 60 * 1000, // Session lasts 1 hour
    httpOnly: true,
    secure:      false, // Set true in production with HTTPS
  }
}));

const users = [];

// LOGIN – sets session
app.post('/login', async function(req, res) {
```



```
const { username, password } = req.body;
const user = users.find(u => u.username === username);
if (!user) return res.status(401).json({ error: 'User not found' });

const valid = await bcrypt.compare(password, user.password);
if (!valid) return res.status(401).json({ error: 'Wrong password' });

// Store user info in session
req.session.userId = user.username;
req.session.loggedIn = true;
res.json({ message: 'Logged in as ' + username });
});

// LOGOUT – destroys session
app.get('/logout', function(req, res) {
  req.session.destroy((err) => {
    if (err) return res.status(500).json({ error: 'Logout failed' });
    res.clearCookie('connect.sid'); // Clear the session cookie
    res.json({ message: 'Logged out successfully!' });
  });
});

// PROTECTED ROUTE – only accessible when logged in
app.get('/dashboard', function(req, res) {
  if (!req.session.loggedIn) {
    return res.status(401).json({ error: 'Please log in first.' });
  }
  res.json({ message: 'Welcome to Dashboard, ' + req.session.userId });
});

app.listen(3000);
```

8.8 Authentication Middleware — Protecting Routes

Instead of checking `req.session.loggedIn` in every protected route, we can create a reusable middleware function:

```
// Middleware to check if user is authenticated
function isAuthenticated(req, res, next) {
  if (req.session && req.session.loggedIn) {
    next(); // User is logged in – allow access
  } else {
    res.status(401).json({ error: 'Unauthorised. Please log in.' });
  }
}
```

```
// Apply middleware to protected routes
app.get('/profile', isAuthenticated, (req, res) => {
  res.json({ message: 'Profile page for ' + req.session.userId });
});

app.get('/dashboard', isAuthenticated, (req, res) => {
  res.json({ message: 'Dashboard for ' + req.session.userId });
});

app.delete('/account', isAuthenticated, (req, res) => {
  res.json({ message: 'Account deletion requested by ' +
    req.session.userId });
});
```

8.9 Complete Authentication Flow — Summary

Step	Action	What Happens
1	User Registers	Server receives credentials, hashes password with bcrypt, stores username + hash in database.
2	User Logs In	Server finds user by username, uses bcrypt.compare() to verify password against stored hash.
3	Session Created	On successful login, server creates a session and sends a session ID cookie to the browser.
4	Subsequent Requests	Browser sends session ID cookie with every request. Server looks up session to identify user.
5	Protected Routes	Middleware checks if session exists and is valid before allowing access to protected routes.
6	User Logs Out	Server destroys the session. Session cookie is cleared from the browser.

9. Exam Practice Questions

Short Answer Questions

1. What is NoSQL? List any two advantages over SQL databases.
2. Differentiate between a Collection and a Document in MongoDB.
3. What is BSON? How is it different from JSON?
4. What is the purpose of the `_id` field in MongoDB? What data type is it by default?
5. Write a MongoDB query to find all books where price is greater than 300.
6. Write a MongoDB query to find all books that belong to 'Fiction' or 'Fantasy' genre.
7. What is the difference between `updateOne()` and `updateMany()`?
8. What does the `$set` operator do in MongoDB? Give an example.
9. What does the `$inc` operator do? How is it different from `$set`?
10. What is body parsing in Express? Explain the `body-parser` package and the built-in Express middleware (`express.json` and `express.urlencoded`) used for it.
11. What is a cookie? How do you set and read a cookie in Express?
12. What is the difference between authentication and authorisation?
13. Why should passwords never be stored as plain text? What should be used instead?
14. What is the purpose of `bcrypt.compare()` in the login process?
15. What is `express-session`? How does it help in maintaining user login state?

Long Answer Questions

1. Explain the MongoDB CRUD operations with proper syntax and examples for each operation (`insertOne`, `insertMany`, `find`, `findOne`, `updateOne`, `updateMany`, `deleteOne`, `deleteMany`).
2. Write a complete Node.js program to connect to MongoDB and perform all CRUD operations on a 'students' collection.
3. Explain how to connect Node.js to a MySQL database using `mysql2`. Write a program to insert, update, delete, and select records from a table.
4. What are cookies? Explain how to set, read, and delete cookies in an Express application with a complete example.
5. Explain the complete user authentication flow in a Node.js application including user registration with password hashing (`bcrypt`) and login with session management (`express-session`). Write the code for each step.
6. Differentiate between SQL and NoSQL databases in detail. When would you choose MongoDB over MySQL?